

Assignment-2 (MAT 3141)

Applied Statistics & Probability

```
In [43]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, linregress

plt.rcParams['figure.figsize']=(7,5)
```

1(a)

```
In [44]: training_time = np.array([2,3,4,5,6,7,8,9])
accuracy = np.array([60,65,68,72,78,82,88,91])

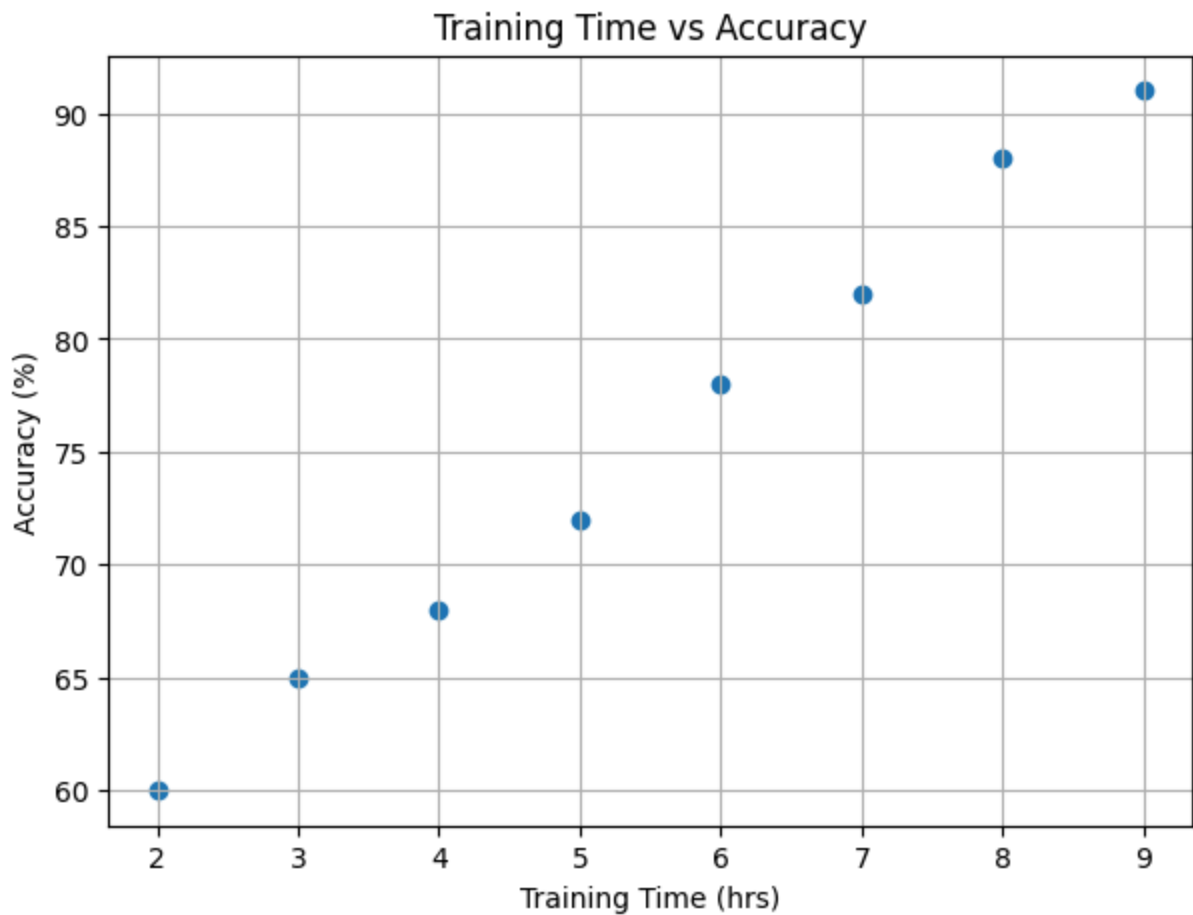
r, p = pearsonr(training_time, accuracy)

print("Karl Pearson Correlation Coefficient =", round(r,4))

if r > 0.8:
    print("Interpretation: Very strong positive relationship.")
elif r > 0.5:
    print("Interpretation: Moderate positive relationship.")
else:
    print("Interpretation: Weak relationship.")

plt.scatter(training_time, accuracy)
plt.xlabel("Training Time (hrs)")
plt.ylabel("Accuracy (%)")
plt.title("Training Time vs Accuracy")
plt.grid(True)
plt.show()
```

Karl Pearson Correlation Coefficient = 0.9974
Interpretation: Very strong positive relationship.



1(b)

```
In [45]: gpu_cost = np.array([15,15,20,25,30,35])
revenue = np.array([25,35,45,50,60,72])

slope, intercept, r_value, p_value, std_err = linregress(gpu_cost, revenue)

print("Regression Equation:")
print(f"Revenue = {intercept:.4f} + {slope:.4f} × GPU Cost")

predicted = intercept + slope*28
print("\nPredicted Revenue for GPU Cost = 28 thousand TK")
print("Predicted Revenue =", round(predicted,2), "thousand TK")

plt.scatter(gpu_cost, revenue, label='Data')
x = np.linspace(min(gpu_cost), max(gpu_cost),100)
plt.plot(x, intercept+slope*x, label='Regression Line')
plt.xlabel("GPU Cost (thousand TK)")
plt.ylabel("Revenue (thousand TK)")
plt.title("Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```

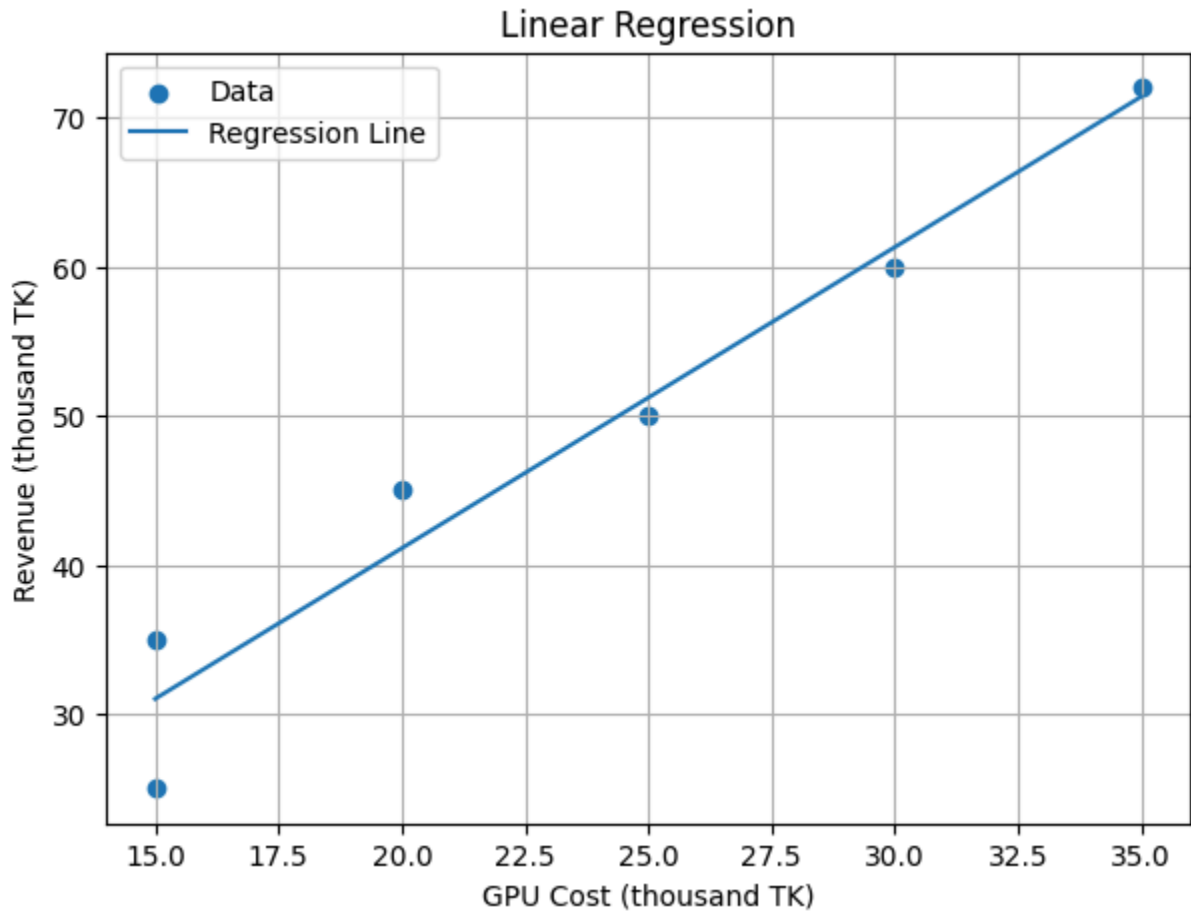
```
print("R-squared =", round(r_value**2,4))
print("Interpretation: Revenue increases as GPU cost increases.")
```

Regression Equation:

Revenue = 0.7000 + 2.0200 × GPU Cost

Predicted Revenue for GPU Cost = 28 thousand TK

Predicted Revenue = 57.26 thousand TK



R-squared = 0.9506

Interpretation: Revenue increases as GPU cost increases.

2(a)

```
In [46]: team_x = np.array([12,13,12,11,14,13])
team_y = np.array([8,10,12,15,18,20])
```

```
def variability_stats(data):
    q1 = np.percentile(data,25)
    q3 = np.percentile(data,75)
    qd = (q3-q1)/2
    cq = (q3-q1)/(q3+q1)
    sd = np.std(data, ddof=1)
    cv = (sd/np.mean(data))*100
    return qd,cq,sd,cv
```

```

qd_x, cq_d_x, sd_x, cv_x = variability_stats(team_x)
qd_y, cq_d_y, sd_y, cv_y = variability_stats(team_y)

print(f"{'Measure':<10}{'Team X':<15}{'Team Y':<15}")
print("-"*40)
print(f"{'QD':<10}{qd_x:<15.3f}{qd_y:<15.3f}")
print(f"{'CQD':<10}{cq_d_x:<15.3f}{cq_d_y:<15.3f}")
print(f"{'SD':<10}{sd_x:<15.3f}{sd_y:<15.3f}")
print(f"{'CV (%)':<10}{cv_x:<15.3f}{cv_y:<15.3f}")

if cv_x < cv_y:
    print("\nTeam X is more consistent because it has lower variability.")
else:
    print("\nTeam Y is more consistent because it has lower variability.")

print("Importance: Consistent sprint completion improves planning, predictabil

```

Measure	Team X	Team Y
QD	0.500	3.375
CQD	0.040	0.243
SD	1.049	4.665
CV (%)	8.390	33.726

Team X is more consistent because it has lower variability.
Importance: Consistent sprint completion improves planning, predictability, and Agile team performance.

2(b)

```

In [47]: moisture = np.array([35,36,37,35,38,39,36,37,90,38])

plt.boxplot(moisture)
plt.title("Soil Moisture Box Plot")
plt.ylabel("Moisture (%)")
plt.grid(True)
plt.show()

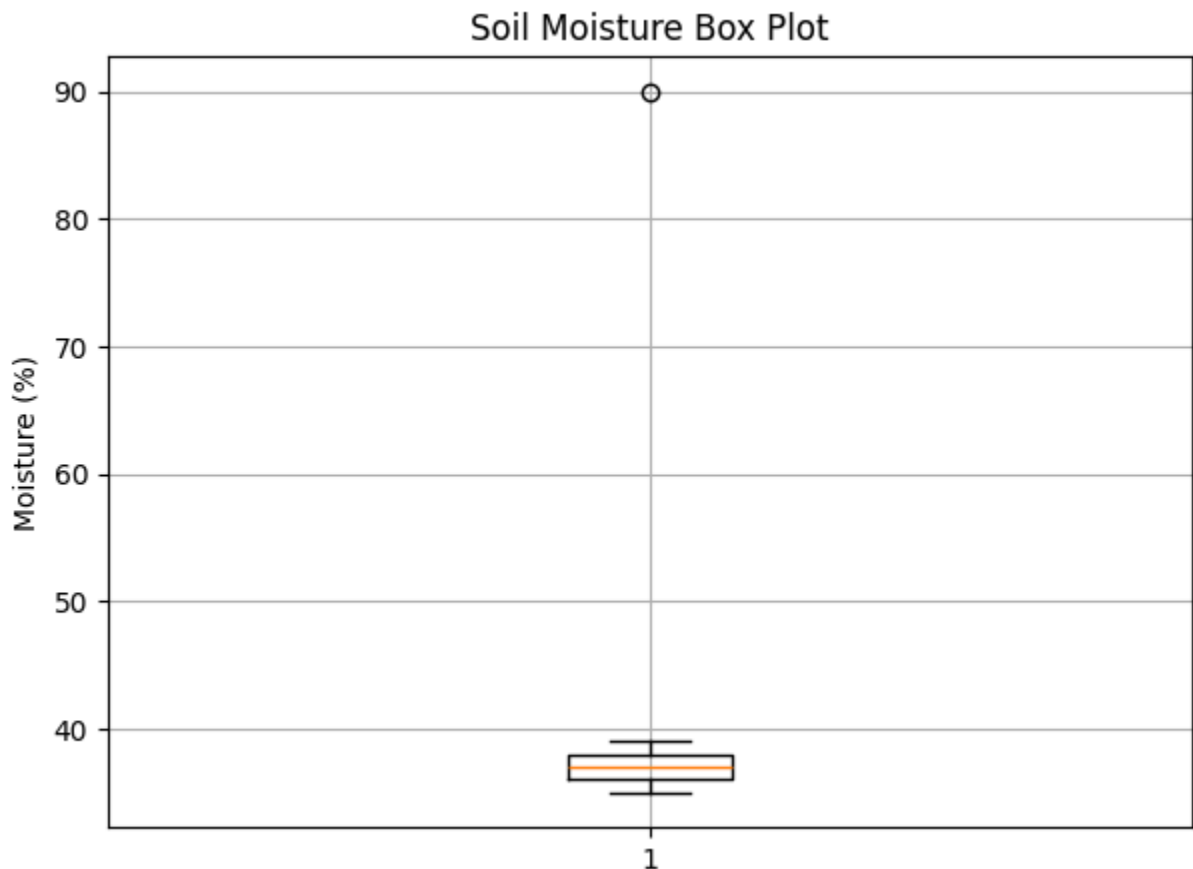
q1 = np.percentile(moisture,25)
q3 = np.percentile(moisture,75)
iqr = q3 - q1

lower = q1 - 1.5*iqr
upper = q3 + 1.5*iqr
outliers = moisture[(moisture < lower) | (moisture > upper)]

print("Q1 =", q1)
print("Q3 =", q3)
print("IQR =", iqr)
print("Lower Bound =", lower)
print("Upper Bound =", upper)

```

```
print("Outliers =", outliers)
print("Interpretation: Abnormal sensor readings can mislead IoT systems, causing incorrect irrigation or decision making.")
```



Q1 = 36.0

Q3 = 38.0

IQR = 2.0

Lower Bound = 33.0

Upper Bound = 41.0

Outliers = [90]

Interpretation: Abnormal sensor readings can mislead IoT systems, causing incorrect irrigation or decision making.

This assignment applied statistical concepts using Python programming.

Correlation and scatter plot analysis showed a strong positive relationship between training time and model accuracy. Linear regression was used to model and predict AI service revenue based on GPU cost. Measures of dispersion (QD, CQD, SD, and CV) were used to compare the consistency of software development teams, while box plot and IQR analysis were used to detect outliers in IoT sensor data. The results demonstrate the importance of statistical analysis, visualization, and predictive modeling in data-driven decision-making.